

TechNest

API Reference Documentation

Backend REST API

Base URL http://localhost:5000	Content-Type application/json
Auth Method JWT Bearer / HttpOnly Cookie	Rate Limit 100 req / 15 min per IP

Prepared by:
Zeina Fahim
Nada Khaled
Alya Mandour
Sama Samer
Miriam Amgad
Nadine Ghazy
Zeina Hesham

Overview

TechNest is a campus facility management platform that allows community members to submit maintenance tickets, facility managers to assign and track them, and workers to resolve issues. This document describes every REST API endpoint exposed by the backend.

Base URL

```
http://localhost:5000
```

Authentication

The API supports two token delivery mechanisms:

- HTTP-only Cookie — set automatically on login (web clients).
- Authorization Header — Bearer token for mobile/API clients.

```
Authorization: Bearer <jwt_token>
```

Roles & Access Levels

Role	Identifier	Permissions Summary
Community Member	community_member	Submit tickets, view own tickets & notifications
Facility Manager	facility_manager	Manage all tickets, assign workers, set priority, merge & close tickets
Worker	worker	View assigned tickets, update status, add comments, upload evidence
Admin	admin	All permissions + user management, role assignment

HTTP Status Codes

Code	Status	Description
200	OK	Request succeeded.
201	Created	Resource created successfully.
400	Bad Request	Validation failed or missing required fields.
401	Unauthorized	No token, expired token, or invalid credentials.
403	Forbidden	Authenticated but insufficient role permissions.
404	Not Found	Requested resource does not exist.
409	Conflict	Resource already exists (e.g. duplicate email).

Code	Status	Description
429	Too Many Requests	Rate limit exceeded (100 requests per 15 minutes).
500	Internal Server Error	Unexpected server-side error.

Rate Limiting

Rate Limit Policy

All /api/* routes are limited to 100 requests per 15-minute window, per IP address.

Exceeded limit returns HTTP 429: { "error": "Too many requests, please try again later." }

1. System

Utility endpoints for service health monitoring and diagnostics. These endpoints are unauthenticated.

1.1 GET /api/health

DESCRIPTION

Returns the current health status of the API server, confirming the service is live and responsive.

REQUEST HEADERS

Header	Value	Required
Content-Type	application/json	No

SUCCESS RESPONSE — 200 OK

```
{
  "status": "OK",
  "app": "CampusCare API",
  "timestamp": "2026-05-17T10:30:00.000Z"
}
```

1.2 GET /api/db-check

DESCRIPTION

Validates database connectivity by querying the user count. Returns an error if the database is unreachable.

SUCCESS RESPONSE — 200 OK

```
{
  "status": "Success",
  "userCount": 42
}
```

ERROR RESPONSE — 500 INTERNAL SERVER ERROR

```
{
  "status": "Error",
  "message": "Connection refused"
}
```

2. Authentication (/api/auth)

Endpoints for user registration, authentication, session management, and password operations. JWT tokens are valid for 7 days.

2.1 POST /api/auth/register

DESCRIPTION

Registers a new user account. Returns a JWT token and sets an HttpOnly session cookie upon success.

REQUEST HEADERS

Header	Value	Required
Content-Type	application/json	Yes

REQUEST BODY

Parameter	Type	Required	Description
name	String	Required	Full name of the user.
email	String	Required	Valid email address. Must be unique.
password	String	Required	Min 8 characters; must contain a letter and a number.
role	String	Optional	One of: community_member, facility_manager, worker, admin. Defaults to "community_member".

SUCCESS RESPONSE — 201 CREATED

```
{
  "message": "Account created successfully.",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "name": "Alice Johnson",
    "email": "alice@campus.edu",
    "role": "community_member",
    "createdAt": "2026-05-17T10:00:00.000Z"
  }
}
```

ERROR RESPONSES

HTTP Status	Error JSON
400 Bad Request	{ "errors": [{ "msg": "Password must be at least 8 characters.", "path": "password" }] }
409 Conflict	{ "error": "An account with this email already exists." }
500 Server Error	{ "error": "Internal server error." }

2.2 POST /api/auth/login

DESCRIPTION

Authenticates an existing user with email and password. Returns a JWT token and sets an HttpOnly session cookie.

REQUEST HEADERS

Header	Value	Required
Content-Type	application/json	Yes

REQUEST BODY

Parameter	Type	Required	Description
email	String	Required	Registered email address.
password	String	Required	Account password.

SUCCESS RESPONSE — 200 OK

```
{
  "message": "Logged in successfully.",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "name": "Alice Johnson",
    "email": "alice@campus.edu",
    "role": "community_member"
  }
}
```

ERROR RESPONSES

HTTP Status	Error JSON
400 Bad Request	{ "errors": [{ "msg": "Valid email is required.", "path": "email" }] }
401 Unauthorized	{ "error": "Invalid email or password." }
403 Forbidden	{ "error": "Account deactivated. Contact an administrator." }

2.3 POST /api/auth/logout

DESCRIPTION

Invalidates the current session by revoking the JWT token server-side and clearing the session cookie.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes (or cookie)

SUCCESS RESPONSE — 200 OK

```
{ "message": "Logged out successfully." }
```

ERROR RESPONSES

HTTP Status	Error JSON
401 Unauthorized	{ "error": "Authentication required. Please log in." }

2.4 GET /api/auth/me

DESCRIPTION

Returns the profile of the currently authenticated user.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes (or cookie)

SUCCESS RESPONSE — 200 OK

```
{
  "user": {
    "id": 1,
    "name": "Alice Johnson",
    "email": "alice@campus.edu",
    "role": "community_member",
    "isActive": true,
    "createdAt": "2026-05-17T10:00:00.000Z"
  }
}
```

ERROR RESPONSES

HTTP Status	Error JSON
401 Unauthorized	{ "error": "Authentication required. Please log in." }
401 Unauthorized	{ "error": "Session expired. Please log in again." }

2.5 GET /api/auth/workers

DESCRIPTION

Returns a list of all active workers. Restricted to facility managers only.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes – facility_manager role

SUCCESS RESPONSE — 200 OK

```
{
  "workers": [
    { "id": 3, "name": "Bob Smith", "email": "bob@campus.edu", "role": "worker" },
    { "id": 7, "name": "Carol Davis", "email": "carol@campus.edu", "role": "worker" }
  ]
}
```

ERROR RESPONSES

HTTP Status	Error JSON
401 Unauthorized	{ "error": "Authentication required. Please log in." }
403 Forbidden	{ "error": "Access denied. Required role: facility_manager" }

2.6 POST /api/auth/forgot-password

DESCRIPTION

Initiates a password reset flow. Generates a one-time reset token valid for 1 hour. Always returns the same success message to prevent email enumeration. In development mode, the raw token is included in the response.

REQUEST HEADERS

Header	Value	Required
Content-Type	application/json	Yes

REQUEST BODY

Parameter	Type	Required	Description
email	String	Required	Email address associated with the account.

SUCCESS RESPONSE — 200 OK

```
{
  "message": "If this email is registered, a reset link has been sent.",
  "devToken": "a3f2c1..." // Development only – omitted in production
}
```

```
| }
```

ERROR RESPONSES

HTTP Status	Error JSON
400 Bad Request	{ "error": "Email is required." }

2.7 POST /api/auth/reset-password

DESCRIPTION

Resets a user's password using a valid, unexpired reset token obtained from the forgot-password endpoint.

REQUEST HEADERS

Header	Value	Required
Content-Type	application/json	Yes

REQUEST BODY

Parameter	Type	Required	Description
token	String	Required	Raw reset token received from forgot-password.
newPassword	String	Required	New password. Minimum 8 characters.

SUCCESS RESPONSE — 200 OK

```
| { "message": "Password reset successfully. Please log in." }
```

ERROR RESPONSES

HTTP Status	Error JSON
400 Bad Request	{ "error": "Token and new password are required." }
400 Bad Request	{ "error": "Invalid or expired reset token." }
400 Bad Request	{ "error": "Password must be at least 8 characters." }

2.8 PUT /api/auth/change-password

DESCRIPTION

Allows an authenticated user to update their own password by providing their current password for verification.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes (or cookie)
Content-Type	application/json	Yes

REQUEST BODY

Parameter	Type	Required	Description
currentPassword	String	Required	The user's existing password.
newPassword	String	Required	New password. Minimum 8 characters.

SUCCESS RESPONSE — 200 OK

```
{ "message": "Password changed successfully." }
```

ERROR RESPONSES

HTTP Status	Error JSON
400 Bad Request	{ "error": "Both fields are required." }
400 Bad Request	{ "error": "Must be at least 8 characters." }
401 Unauthorized	{ "error": "Current password is incorrect." }

3. Tickets (/api/tickets)

Core facility management endpoints. All ticket routes require authentication. Role-based access controls are enforced per endpoint.

Valid Enumerations
Categories: electrical | plumbing | cleaning | furniture | other
Statuses: pending | in_progress | resolved | closed
Priorities: low | medium | high

3.1 POST /api/tickets

DESCRIPTION
Submits a new maintenance ticket. Restricted to community members. An optional image attachment can be uploaded via multipart/form-data.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes – community_member role
Content-Type	multipart/form-data	Yes (for file upload) or application/json

REQUEST BODY

Parameter	Type	Required	Description
title	String	Required	Short descriptive title of the issue.
description	String	Required	Detailed description of the problem.
category	String	Required	Issue category (electrical, plumbing, cleaning, furniture, other).
building	String	Required	Building name or identifier.
floor	String	Required	Floor number or identifier.
roomNumber	String	Required	Room number or identifier.
image	File	Optional	Image of the issue. Sent as multipart form-data field named 'image'.

SUCCESS RESPONSE — 201 CREATED

```
{
  "message": "Ticket submitted successfully.",
  "ticket": {
    "id": 12,
    "title": "Broken AC Unit",
    "description": "The AC unit in room 204 is not cooling.",
```

```
{
  "category": "electrical",
  "status": "pending",
  "priority": "medium",
  "building": "Engineering Block",
  "floor": "2",
  "roomNumber": "204",
  "imageUrl": "/uploads/image-1715942400000.jpg",
  "submittedBy": { "id": 1, "name": "Alice Johnson", "email": "alice@campus.edu" },
  "createdAt": "2026-05-17T10:00:00.000Z"
}
```

ERROR RESPONSES

HTTP Status	Error JSON
400 Bad Request	{ "errors": [{ "msg": "Title is required.", "path": "title" }] }
401 Unauthorized	{ "error": "Authentication required. Please log in." }
403 Forbidden	{ "error": "Access denied. Required role: community_member" }

3.2 GET /api/tickets

DESCRIPTION

Retrieves all tickets with optional filtering and pagination. Restricted to facility managers and admins. Results are ordered by priority (descending) then creation date.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes – facility_manager or admin role

QUERY PARAMETERS

Parameter	Type	Required	Description
status	String	Optional	Filter by status: pending, in_progress, resolved, closed.
category	String	Optional	Filter by category: electrical, plumbing, cleaning, furniture, other.
priority	String	Optional	Filter by priority: low, medium, high.
page	Integer	Optional	Page number for pagination. Defaults to 1.
limit	Integer	Optional	Results per page. Defaults to 20.

SUCCESS RESPONSE — 200 OK

```
{
```

```

    "tickets": [
      {
        "id": 12,
        "title": "Broken AC Unit",
        "status": "pending",
        "priority": "medium",
        "category": "electrical",
        "building": "Engineering Block",
        "submittedBy": { "id": 1, "name": "Alice Johnson", "email": "alice@campus.edu" },
        "assignedTo": null,
        "_count": { "comments": 2, "mergedTickets": 0 }
      },
      {
        "id": 13,
        "title": "Broken AC Unit",
        "status": "pending",
        "priority": "medium",
        "category": "electrical",
        "building": "Engineering Block",
        "submittedBy": { "id": 1, "name": "Alice Johnson", "email": "alice@campus.edu" },
        "assignedTo": null,
        "_count": { "comments": 2, "mergedTickets": 0 }
      }
    ],
    "pagination": { "page": 1, "limit": 20, "total": 45, "pages": 3 }
  }
}
```

ERROR RESPONSES

HTTP Status	Error JSON
401 Unauthorized	{ "error": "Authentication required. Please log in." }
403 Forbidden	{ "error": "Access denied. Required role: facility_manager, admin" }

3.3 GET /api/tickets/my

DESCRIPTION

Returns all tickets submitted by the currently authenticated community member, ordered by creation date descending.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes – community_member role

SUCCESS RESPONSE — 200 OK

```

{
  "tickets": [
    {
      "id": 12,
      "title": "Broken AC Unit",
      "status": "in_progress",
      "priority": "medium",
      "assignedTo": { "id": 3, "name": "Bob Smith" },
      "count": { "comments": 3 },
      "createdAt": "2026-05-17T10:00:00.000Z"
    },
    {
      "id": 13,
      "title": "Broken AC Unit",
      "status": "in_progress",
      "priority": "medium",
      "assignedTo": { "id": 3, "name": "Bob Smith" },
      "count": { "comments": 3 },
      "createdAt": "2026-05-17T10:00:00.000Z"
    }
  ]
}
```

ERROR RESPONSES

HTTP Status	Error JSON
401 Unauthorized	{ "error": "Authentication required. Please log in." }
403 Forbidden	{ "error": "Access denied. Required role: community_member" }

3.4 GET /api/tickets/assigned

DESCRIPTION

Returns all tickets currently assigned to the authenticated worker, ordered by priority then creation date.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes – worker role

SUCCESS RESPONSE — 200 OK

```
{
  "tickets": [
    {
      "id": 12,
      "title": "Broken AC Unit",
      "status": "in_progress",
      "priority": "high",
      "submittedBy": { "id": 1, "name": "Alice Johnson" },
      "_count": { "comments": 1 }
    }
  ]
}
```

ERROR RESPONSES

HTTP Status	Error JSON
401 Unauthorized	{ "error": "Authentication required. Please log in." }
403 Forbidden	{ "error": "Access denied. Required role: worker" }

3.5 GET /api/tickets/:id

DESCRIPTION

Retrieves full details for a single ticket by its numeric ID, including comments, merged ticket relationships, and assigned worker info. Community members may only access their own tickets.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes

URL PARAMETERS

Parameter	Type	Description
:id	Integer	The unique numeric ID of the ticket.

SUCCESS RESPONSE — 200 OK

```
{
  "ticket": {
    "id": 12,
    "title": "Broken AC Unit",
    "description": "The AC unit in room 204 is not cooling.",
    "category": "electrical",
    "status": "in_progress",
    "priority": "high",
    "building": "Engineering Block",
    "floor": "2",
    "roomNumber": "204",
    "imageUrl": "/uploads/image-1715942400000.jpg",
    "evidenceUrl": null,
    "submittedBy": { "id": 1, "name": "Alice Johnson", "email": "alice@campus.edu" },
    "assignedTo": { "id": 3, "name": "Bob Smith", "email": "bob@campus.edu" },
    "mergedInto": null,
    "mergedTickets": [],
    "comments": [
      { "id": 5, "content": "Technician dispatched.", "createdAt": "2026-05-17T11:00:00.000Z",
        "author": { "id": 3, "name": "Bob Smith", "role": "worker" } }
    ],
    "createdAt": "2026-05-17T10:00:00.000Z"
  }
}
```

ERROR RESPONSES

HTTP Status	Error JSON
401 Unauthorized	{ "error": "Authentication required. Please log in." }
403 Forbidden	{ "error": "Access denied." }
404 Not Found	{ "error": "Ticket not found." }

3.6 PUT /api/tickets/:id/status

DESCRIPTION

Updates the status of a ticket. Accessible by facility managers and workers.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes — facility_manager or worker role
Content-Type	application/json	Yes

URL PARAMETERS

Parameter	Type	Description
:id	Integer	The unique numeric ID of the ticket.

REQUEST BODY

Parameter	Type	Required	Description
status	String	Required	New status value: pending, in_progress, resolved, or closed.

SUCCESS RESPONSE — 200 OK

```
{
  "message": "Status updated.",
  "ticket": { "id": 12, "status": "resolved", ... }
}
```

ERROR RESPONSES

HTTP Status	Error JSON
400 Bad Request	{ "errors": [{ "msg": "Invalid status.", "path": "status" }] }
401 Unauthorized	{ "error": "Authentication required. Please log in." }
403 Forbidden	{ "error": "Access denied. Required role: facility_manager, worker" }

3.7 PUT /api/tickets/:id/priority

DESCRIPTION

Sets the priority level of a ticket. Restricted to facility managers.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes — facility_manager role

Header	Value	Required
Content-Type	application/json	Yes

URL PARAMETERS

Parameter	Type	Description
:id	Integer	The unique numeric ID of the ticket.

REQUEST BODY

Parameter	Type	Required	Description
priority	String	Required	Priority level: low, medium, or high.

SUCCESS RESPONSE — 200 OK

```
{
  "message": "Priority updated.",
  "ticket": { "id": 12, "priority": "high", ... }
}
```

ERROR RESPONSES

HTTP Status	Error JSON
400 Bad Request	{ "errors": [{ "msg": "Invalid priority.", "path": "priority" }] }
403 Forbidden	{ "error": "Access denied. Required role: facility_manager" }

3.8 PUT /api/tickets/:id/assign

DESCRIPTION

Assigns a ticket to a specific active worker. Automatically sets the ticket status to `in_progress`. Sends notifications to both the assigned worker and the ticket submitter.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes — facility_manager role
Content-Type	application/json	Yes

URL PARAMETERS

Parameter	Type	Description
:id	Integer	The unique numeric ID of the ticket.

REQUEST BODY

Parameter	Type	Required	Description
workerId	Integer	Required	The numeric ID of the active worker to assign.

SUCCESS RESPONSE — 200 OK

```
{
  "message": "Ticket assigned successfully.",
  "ticket": {
    "id": 12,
    "status": "in_progress",
    "assignedTo": { "id": 3, "name": "Bob Smith", "email": "bob@campus.edu" },
    "submittedBy": { "id": 1, "name": "Alice Johnson" }
  }
}
```

ERROR RESPONSES

HTTP Status	Error JSON
400 Bad Request	{ "errors": [{ "msg": "Worker ID is required.", "path": "workerId" }] }
403 Forbidden	{ "error": "Access denied. Required role: facility_manager" }
404 Not Found	{ "error": "Worker not found or inactive." }

3.9 PUT /api/tickets/:id/close

DESCRIPTION

Closes a ticket by setting its status to closed. Notifies the original submitter. Restricted to facility managers.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes — facility_manager role

URL PARAMETERS

Parameter	Type	Description
:id	Integer	The unique numeric ID of the ticket to close.

SUCCESS RESPONSE — 200 OK

```
{
  "message": "Ticket closed.",
  "ticket": { "id": 12, "status": "closed", ... }
}
```

```
} }
```

ERROR RESPONSES

HTTP Status	Error JSON
401 Unauthorized	{ "error": "Authentication required. Please log in." }
403 Forbidden	{ "error": "Access denied. Required role: facility_manager" }

3.10 POST /api/tickets/:id/merge

DESCRIPTION

Merges one or more duplicate tickets into a target ticket. Merged tickets are set to closed and linked to the target. Restricted to facility managers.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes – facility_manager role
Content-Type	application/json	Yes

URL PARAMETERS

Parameter	Type	Description
:id	Integer	The ID of the target ticket to merge into.

REQUEST BODY

Parameter	Type	Required	Description
ticketIds	Integer[]	Required	Array of ticket IDs to merge into the target. The target ID is automatically excluded if present.

SUCCESS RESPONSE — 200 OK

```
{
  "message": "2 ticket(s) merged into #12.",
  "ticket": {
    "id": 12,
    "mergedTickets": [
      { "id": 8, "title": "AC Issue Room 204" },
      { "id": 9, "title": "No cooling in 2nd floor" }
    ]
  }
}
```

ERROR RESPONSES

HTTP Status	Error JSON
400 Bad Request	{ "error": "Provide ticketIds array to merge." }
403 Forbidden	{ "error": "Access denied. Required role: facility_manager" }

3.11 POST /api/tickets/:id/comments

DESCRIPTION

Adds a comment to a ticket. Restricted to workers and facility managers. Automatically notifies the ticket submitter if the commenter is a different user.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes – worker or facility_manager role
Content-Type	application/json	Yes

URL PARAMETERS

Parameter	Type	Description
:id	Integer	The unique numeric ID of the ticket.

REQUEST BODY

Parameter	Type	Required	Description
content	String	Required	The text body of the comment. Cannot be empty.

SUCCESS RESPONSE — 201 CREATED

```
{
  "message": "Comment added.",
  "comment": {
    "id": 7,
    "content": "Parts ordered. Repair scheduled for tomorrow.",
    "createdAt": "2026-05-17T14:00:00.000Z",
    "author": { "id": 3, "name": "Bob Smith", "role": "worker" }
  }
}
```

ERROR RESPONSES

HTTP Status	Error JSON
400 Bad Request	{ "error": "Comment content is required." }

HTTP Status	Error JSON
403 Forbidden	{ "error": "Access denied. Required role: worker, facility_manager" }

3.12 POST /api/tickets/:id/evidence

DESCRIPTION

Uploads a photo as completion evidence and automatically marks the ticket status as resolved. Restricted to workers. Notifies the ticket submitter upon resolution.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes — worker role
Content-Type	multipart/form-data	Yes

URL PARAMETERS

Parameter	Type	Description
:id	Integer	The unique numeric ID of the ticket.

REQUEST BODY (MULTIPART/FORM-DATA)

Field	Type	Required	Description
evidence	File	Required	Image file serving as proof of resolution (field name: 'evidence').

SUCCESS RESPONSE — 200 OK

```
{
  "message": "Evidence uploaded. Ticket marked as resolved.",
  "ticket": {
    "id": 12,
    "status": "resolved",
    "evidenceUrl": "/uploads/evidence-1715950000000.jpg"
  }
}
```

ERROR RESPONSES

HTTP Status	Error JSON
400 Bad Request	{ "error": "Evidence photo is required." }
403 Forbidden	{ "error": "Access denied. Required role: worker" }

3.13 DELETE /api/tickets/:id

DESCRIPTION

Permanently deletes a ticket and all associated comments and notifications. Cascades removal of merge links. Restricted to facility managers and admins.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes – facility_manager or admin role

URL PARAMETERS

Parameter	Type	Description
:id	Integer	The unique numeric ID of the ticket to delete.

SUCCESS RESPONSE — 200 OK

```
{ "message": "Ticket deleted." }
```

ERROR RESPONSES

HTTP Status	Error JSON
401 Unauthorized	{ "error": "Authentication required. Please log in." }
403 Forbidden	{ "error": "Access denied. Required role: facility_manager, admin" }

4. Manager (/api/manager)

Workforce management endpoints for facility managers and admins to monitor and manage worker accounts.

4.1 GET /api/manager/workers

DESCRIPTION

Returns a list of all worker accounts, including their active status and number of assigned tickets. Accessible by facility managers and admins.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes – facility_manager or admin role

SUCCESS RESPONSE — 200 OK

```
{
  "workers": [
    {
      "id": 3,
      "name": "Bob Smith",
      "email": "bob@campus.edu",
      "isActive": true,
      "createdAt": "2026-01-10T08:00:00.000Z",
      "_count": { "assignedTickets": 5 }
    }
  ]
}
```

ERROR RESPONSES

HTTP Status	Error JSON
401 Unauthorized	{ "error": "Authentication required. Please log in." }
403 Forbidden	{ "error": "Access denied. Required role: facility_manager, admin" }

4.2 PUT /api/manager/workers/:id/status

DESCRIPTION

Toggles the active/inactive status of a worker account. An inactive worker cannot log in or be assigned new tickets.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes – facility_manager or admin role

URL PARAMETERS

Parameter	Type	Description
:id	Integer	The unique numeric ID of the worker.

SUCCESS RESPONSE — 200 OK

```
{
  "message": "Worker deactivated.",
  "worker": { "id": 3, "name": "Bob Smith", "isActive": false }
}
```

ERROR RESPONSES

HTTP Status	Error JSON
401 Unauthorized	{ "error": "Authentication required. Please log in." }
403 Forbidden	{ "error": "Access denied. Required role: facility_manager, admin" }
404 Not Found	{ "error": "Worker not found." }

5. Admin (/api/admin)

Platform-wide user administration endpoints. All routes require the admin role.

5.1 GET /api/admin/users

DESCRIPTION

Retrieves all registered users across all roles, ordered by registration date descending.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes – admin role

SUCCESS RESPONSE — 200 OK

```
{
  "users": [
    {
      "id": 1,
      "name": "Alice Johnson",
      "email": "alice@campus.edu",
      "role": "community_member",
      "isActive": true,
      "createdAt": "2026-05-17T10:00:00.000Z"
    }
  ]
}
```

ERROR RESPONSES

HTTP Status	Error JSON
401 Unauthorized	{ "error": "Authentication required. Please log in." }
403 Forbidden	{ "error": "Access denied. Required role: admin" }

5.2 PUT /api/admin/users/:id/status

DESCRIPTION

Toggles the active/inactive status of any user account. Deactivated users cannot authenticate.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes – admin role

URL PARAMETERS

Parameter	Type	Description
:id	Integer	The unique numeric ID of the user.

SUCCESS RESPONSE — 200 OK

```
{
  "message": "User deactivated.",
  "user": { "id": 1, "name": "Alice Johnson", "role": "community_member", "isActive": false }
}
```

ERROR RESPONSES

HTTP Status	Error JSON
401 Unauthorized	{ "error": "Authentication required. Please log in." }
403 Forbidden	{ "error": "Access denied. Required role: admin" }
404 Not Found	{ "error": "User not found." }

5.3 PUT /api/admin/users/:id/role

DESCRIPTION

Assigns or changes the role of a user. Accepts one of the four valid platform roles.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes — admin role
Content-Type	application/json	Yes

URL PARAMETERS

Parameter	Type	Description
:id	Integer	The unique numeric ID of the user.

REQUEST BODY

Parameter	Type	Required	Description
role	String	Required	New role: <code>community_member</code> , <code>facility_manager</code> , <code>worker</code> , or <code>admin</code> .

SUCCESS RESPONSE — 200 OK

```
{
  "message": "Role updated.",
  "user": {
```

```
{
  "id": 1,
  "name": "Alice Johnson",
  "email": "alice@campus.edu",
  "role": "facility_manager"
}
```

ERROR RESPONSES

HTTP Status	Error JSON
400 Bad Request	{ "error": "Role must be one of: community_member, facility_manager, worker, admin" }
401 Unauthorized	{ "error": "Authentication required. Please log in." }
403 Forbidden	{ "error": "Access denied. Required role: admin" }

6. Notifications (/api/notifications)

Endpoints for managing in-app notifications. Notifications are automatically generated by the system on ticket events such as status changes, assignments, comments, and resolutions. All routes require authentication.

6.1 GET /api/notifications

DESCRIPTION

Returns the 50 most recent notifications for the authenticated user, ordered by creation date descending.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes

SUCCESS RESPONSE — 200 OK

```
{
  "notifications": [
    {
      "id": 15,
      "message": "Your ticket \"Broken AC Unit\" has been assigned to Bob Smith.",
      "isRead": false,
      "createdAt": "2026-05-17T11:30:00.000Z",
      "ticket": { "id": 12, "title": "Broken AC Unit" }
    }
  ]
}
```

ERROR RESPONSES

HTTP Status	Error JSON
401 Unauthorized	{ "error": "Authentication required. Please log in." }

6.2 PUT /api/notifications/read-all

DESCRIPTION

Marks all unread notifications belonging to the authenticated user as read in a single operation.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes

SUCCESS RESPONSE — 200 OK

```
{ "message": "All notifications marked as read." }
```

ERROR RESPONSES

HTTP Status	Error JSON
401 Unauthorized	{ "error": "Authentication required. Please log in." }

6.3 PUT /api/notifications/:id/read

DESCRIPTION

Marks a single notification as read. Users may only mark their own notifications.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes

URL PARAMETERS

Parameter	Type	Description
:id	Integer	The unique numeric ID of the notification.

SUCCESS RESPONSE — 200 OK

```
{ "message": "Marked as read." }
```

ERROR RESPONSES

HTTP Status	Error JSON
401 Unauthorized	{ "error": "Authentication required. Please log in." }
403 Forbidden	{ "error": "Access denied." }
404 Not Found	{ "error": "Notification not found." }

6.4 DELETE /api/notifications/:id

DESCRIPTION

Permanently deletes a notification. Users may only delete their own notifications.

REQUEST HEADERS

Header	Value	Required
Authorization	Bearer <jwt_token>	Yes

URL PARAMETERS

Parameter	Type	Description
:id	Integer	The unique numeric ID of the notification.

SUCCESS RESPONSE — 200 OK

```
{ "message": "Notification deleted." }
```

ERROR RESPONSES

HTTP Status	Error JSON
401 Unauthorized	{ "error": "Authentication required. Please log in." }
403 Forbidden	{ "error": "Access denied." }
404 Not Found	{ "error": "Notification not found." }

7. Endpoint Quick-Reference

Complete list of all API endpoints with their required authentication role.

Method	Endpoint	Required Role	Description
GET	/api/health	None	API health check
GET	/api/db-check	None	Database connectivity check
POST	/api/auth/register	None	Register new account
POST	/api/auth/login	None	Login and obtain JWT
POST	/api/auth/logout	Any authenticated	Invalidate session token
GET	/api/auth/me	Any authenticated	Get current user profile
GET	/api/auth/workers	facility_manager	List active workers
POST	/api/auth/forgot-password	None	Initiate password reset
POST	/api/auth/reset-password	None	Complete password reset
PUT	/api/auth/change-password	Any authenticated	Change own password
POST	/api/tickets	community_member	Submit new ticket
GET	/api/tickets	facility_manager, admin	List all tickets (paginated)
GET	/api/tickets/my	community_member	List own tickets
GET	/api/tickets/assigned	worker	List assigned tickets
GET	/api/tickets/:id	Any authenticated	Get ticket details
PUT	/api/tickets/:id/status	facility_manager, worker	Update ticket status
PUT	/api/tickets/:id/priority	facility_manager	Set ticket priority
PUT	/api/tickets/:id/assign	facility_manager	Assign ticket to worker
PUT	/api/tickets/:id/close	facility_manager	Close a ticket
POST	/api/tickets/:id/merge	facility_manager	Merge tickets
POST	/api/tickets/:id/comments	worker, facility_manager	Add comment
POST	/api/tickets/:id/evidence	worker	Upload resolution evidence
DELETE	/api/tickets/:id	facility_manager, admin	Delete ticket

Method	Endpoint	Required Role	Description
GET	/api/manager/workers	facility_manager, admin	List all workers
PUT	/api/manager/workers/:id/status	facility_manager, admin	Toggle worker status
GET	/api/admin/users	admin	List all users
PUT	/api/admin/users/:id/status	admin	Toggle user status
PUT	/api/admin/users/:id/role	admin	Assign role to user
GET	/api/notifications	Any authenticated	Get own notifications
PUT	/api/notifications/read-all	Any authenticated	Mark all as read
PUT	/api/notifications/:id/read	Any authenticated	Mark notification as read
DELETE	/api/notifications/:id	Any authenticated	Delete notification